ALLIANCE UNIVERSITY • SCHOOL OF COMPUTING • DEPARTMENT OF COMPUTER APPLICATIONS

Assignment 3: Database Design Analysis & Normalization

Proposed System: Smart Library Management System (SLMS)

Student Name: Sanjeev Adarsh S
Program / Semester: BCA • Semester III
Course: Database Management Systems (DBMS Lab)
Deliverable: Individual Assignment 3 (PDF)

1. Introduction & Proposed System Overview

The **Smart Library Management System (SLMS)** is an enterprise academic database system designed for Alliance University to automate library administration, bibliographic indexing, multi-copy book tracking, student and faculty membership management, circulation loans, and overdue fine settlements.

A dependable relational database foundation requires rigorous logical modeling. Initial designs often suffer from excessive data redundancy, structural coupling, and operational anomalies. This report conducts a comprehensive normalization analysis of the initial library schema, examines the functional dependencies governing its attributes, demonstrates how poor logical design causes insertion, deletion, and update anomalies, and details the mathematical decomposition into Boyce-Codd Normal Form (BCNF).

2. Initial Unnormalized Schema & Data Redundancy Analysis

Consider an initial, naive unnormalized design where all entity attributes and transactions are collapsed into a single broad relation:

```
UNNORMALIZED_LIBRARY_REGISTRY(
    MemberID, FullName, Email, Phone, MembershipType, MaxBooksAllowed,
    BookID, ISBN, Title, PublicationYear, Price, TotalCopies, AvailableCopies,
    CategoryID, CategoryName, CategoryDesc,
    PublisherID, PublisherName, PublisherCountry, PublisherEmail,
    AuthorNames, StaffID, StaffName, StaffRole,
    LoanID, IssueDate, DueDate, ReturnDate, LoanStatus,
    FineID, FineAmount, PaidAmount, PaymentStatus
)
```

An analysis of this flat relational structure reveals acute **data redundancy** across several operational boundaries:

- **Publisher & Category Duplication:** For every book copy issued, the publisher's country, corporate email, and category description are redundantly recorded. A catalog of 5,000 circulations from Pearson Education duplicates the publisher's metadata 5,000 times.
- **Member Profile Duplication:** An active student borrowing multiple books over a semester repeats their full name, institutional email, phone number, and membership quota across every loan tuple.
- **Catalog Title Repetition:** Highly demanded textbooks (e.g., *"Introduction to Algorithms"*) duplicate title strings, publication years, and list prices across every circulation row.
- **Non-Atomic Multi-Valued Attributes:** Co-authored academic volumes (e.g., Stuart Russell and Peter Norvig for AI) bundle multiple author identities into a single comma-delimited string, violating atomicity.

3. Operational Anomalies Induced by Poor Design

When multiple real-world entities are combined in an unnormalized relation, the database exhibits three critical failure patterns:

3.1 Insertion Anomaly

An insertion anomaly occurs when valid, independent business facts cannot be recorded without artificially manufacturing unrelated data or violating primary key constraints.

- **Unborrowed Catalog Acquisitions:** When the library acquires a newly released edition of *"Designing Data-Intensive Applications"*, its bibliographic details cannot be inserted into `UNNORMALIZED_LIBRARY_REGISTRY` until a student borrows it. Because `LoanID` or `MemberID` forms part of the candidate key, leaving them `NULL` violates **Entity Integrity**.
- **New Patron Registration:** When a newly admitted BCA student registers for a library card, their demographic profile cannot be saved without simultaneously issuing a book checkout.

3.2 Deletion Anomaly

A deletion anomaly occurs when purging an expired or completed operational transaction unintentionally erases completely unrelated foundational business knowledge.

- **Catalog Loss Upon Loan Clearance:** If an alumni member returns a rare monograph on *"Information Theory"* and the transaction record is archived or deleted, the system destroys the sole surviving record confirming the book's existence, its ISBN, publisher contact, and subject classification.

3.3 Update / Modification Anomaly

An update anomaly arises when redundant copies of a single fact must be maintained across multiple rows. If an update query fails to modify every occurrence, the database enters an inconsistent, corrupt state.

- **Patron Contact Alteration:** If a student changes their contact phone number, dozens of historical loan rows must be located and modified. A partial update leaves conflicting contact numbers for the same student.
- **Publisher Rebranding / Reorganization:** If a publisher updates its customer service email or corporate address, thousands of historical circulation records must be rewritten.

4. Functional Dependencies (FD) Analysis

A functional dependency \(X ightarrow Y\) indicates that the value of attribute set \(X\) uniquely and deterministically establishes the value of attribute set \(Y\). The operational rules of the Smart Library Management System establish the following formal functional dependencies:

FD ID	Functional Dependency Statement	Domain Semantics & Business Rule
FD1	`BookID → ISBN, Title, CategoryID, PublisherID, PublicationYear, TotalCopies, AvailableCopies, Price`	The internal catalog surrogate key uniquely determines all bibliographic, inventory, and pricing properties.
FD2	`ISBN → BookID, Title, PublisherID, Price`	The International Standard Book Number is a universal alternate candidate key for published editions.
FD3	`PublisherID → PublisherName, PublisherCountry, PublisherEmail`	A publishing company's corporate identity and communication points are determined solely by PublisherID.
FD4	`CategoryID → CategoryName, CategoryDesc`	Academic subject classification details depend strictly on CategoryID.
FD5	`AuthorID → AuthorName, Nationality`	Author biographical attributes depend strictly on AuthorID.

FD ID	Functional Dependency Statement	Domain Semantics & Business Rule
FD6	`MemberID → FullName, Email, Phone, MembershipType, MaxBooksAllowed, JoinDate, Status`	A library member's profile, contact information, and borrowing limits are uniquely determined by MemberID.
FD7	`Email → MemberID, FullName, Phone`	Institutional email addresses are globally unique per user account.
FD8	`StaffID → StaffName, StaffRole, Email, HireDate`	Operational staff credentials and administrative roles depend on StaffID.
FD9	`LoanID → BookID, MemberID, StaffID, IssueDate, DueDate, ReturnDate, LoanStatus`	A circulation transaction record uniquely ties one book, one member, and the issuing staff clerk.
FD10	`FineID → LoanID, FineAmount, PaidAmount, PaymentStatus, PaymentDate`	Financial penalty balances are determined by FineID.
FD11	`LoanID → FineID`	Each overdue circulation event maps to at most one penalty assessment record (1:1 relationship).

5. Why Poor Database Design Causes Anomalies

The core theoretical reason poor database designs suffer from anomalies is the violation of relational normalization principles: specifically, **permitting non-trivial functional dependencies where the determinant is not a candidate key**.

When non-key attributes determine other non-key attributes (transitive dependency: \(X ightarrow Y\) and \(Y ightarrow Z\)), or when non-key attributes depend on only a fraction of a composite key (partial dependency), two or more distinct business themes are forced into the same relation. In the unnormalized library registry, catalog management, member demographics, publisher agreements, and loan workflows were merged. Normalization mathematically decomposes these relations into independent tables so that *"every fact is stored exactly once, and every non-key attribute depends upon the key, the whole key, and nothing but the key."*

6. Step-by-Step Normalization Process

6.1 First Normal Form (1NF) • Eliminating Multi-Valued Attributes

Rule: All attributes must contain atomic (scalar) values; repeating groups and multi-valued arrays are strictly prohibited.

Improvement: In the unnormalized table, `AuthorNames` stored comma-separated values (e.g., *"Donald Knuth"* or *"Stuart Russell, Peter Norvig"*). We isolate authors into an independent `authors` entity and create an associative junction table `book_authors(BookID, AuthorID)`. This handles Many-to-Many (\(M:N\)) relationships cleanly and enforces scalar values across all cells.

6.2 Second Normal Form (2NF) • Eliminating Partial Functional Dependencies

Rule: The relation must be in 1NF, and no non-prime attribute may be functionally dependent on a proper subset of any candidate key.

Improvement: In an unnormalized composite candidate key like `(LoanID, BookID, MemberID)`, attributes such as `Title` depend only on `BookID`, while `MemberName` depends only on `MemberID`. We decompose the relation into distinct entity tables (`books`, `members`) and an association table (`loans`) where each attribute depends on the full primary key.

6.3 Third Normal Form (3NF) • Eliminating Transitive Dependencies

Rule: The relation must be in 2NF, and no non-prime attribute may depend transitively on the primary key (\(X ightarrow Y ightarrow Z\)).

Improvement: In the preliminary books relation, `BookID → PublisherID` and `PublisherID → PublisherName, PublisherCountry`. Here, publisher attributes were transitively dependent on `BookID`. Similarly, `BookID → CategoryID → CategoryName`. We extract `publishers` and `categories` into standalone relations, retaining only their surrogate primary keys as foreign keys within `books`.

6.4 Boyce-Codd Normal Form (BCNF) • Candidate Key Determinants

Rule: For every functional dependency \(X ightarrow Y\) in relation \(R\), the determinant \(X\) must be a superkey.

Improvement: In the financial fine ledger, `FineID → LoanID, FineAmount, PaidAmount` and `LoanID → FineID`. Both `FineID` and `LoanID` are valid candidate keys for the fine record. Isolating `fines` as a dedicated relation with `fine_id` as primary key and `loan_id` as a unique foreign key ensures that every determinant is a superkey, satisfying BCNF.

7. Finalized Relational Database Schema Design

The finalized, fully normalized database design for the Smart Library Management System consists of **9 distinct relations**:

Table Name	Primary Key	Foreign Keys	Normal Form & Description
publishers	`publisher_id`	None	BCNF • Publication houses and international contacts
authors	`author_id`	None	BCNF • Author biographical identities and nationalities
categories	`category_id`	None	BCNF • Academic subject disciplines and classifications
books	`book_id`	`category_id`, `publisher_id`	BCNF • Catalog titles, inventory quantities, and pricing
book_authors	`(book_id, author_id)`	`book_id`, `author_id`	BCNF • Junction entity resolving \(M:N\) book-author links
members	`member_id`	None	BCNF • Patron accounts (Student, Faculty, Alumni, Guest)
staff	`staff_id`	None	BCNF • Library administrative and operational personnel
loans	`loan_id`	`book_id`, `member_id`, `staff_id`	BCNF • Circulation checkout, due dates, and return ledger
fines	`fine_id`	`loan_id`	BCNF • Penalty assessment, payment status, and receipts

> ***Outcome & Verification:*** *Normalizing the SLMS schema completely eradicates redundant data storage across transactions, guarantees that books and members can be instantiated independently of loan events, safeguards the catalog against unintended deletion, and ensures that member and publisher updates occur in exactly one place.*